

Django Backend Test – Real-Time Chat Assignment

Objective

Build a **simple real-time chat application** using **Django** that allows **two authenticated users** to exchange messages instantly using **WebSockets**.

This assignment is **backend-focused**. UI can be minimal and purely functional.

Skills Being Evaluated

- Django project & app structure
 - Django authentication
 - Django Channels (WebSockets)
 - Async vs sync handling
 - Message persistence
 - Security & access control
 - Code clarity and best practices
-

Required Tech Stack

- Python 3.11+
 - **Django**
 - **Django REST Framework**
 - **Django Channels**
 - **Redis** (for channel layer)
 - SQLite or PostgreSQL or MySQL
-

Functional Requirements

Authentication

- Use Django's built-in `User` model
 - Only authenticated users can access chat functionality
 - Assume only **two users per chat** (no group chats)
-

Chat Logic

- A chat exists **between two users only**

- Only users involved in the chat can:
 - Connect to the WebSocket
 - Send or receive messages
-

WebSocket Communication

- Implement real-time messaging using **Django Channels**
- Example WebSocket URL structure:

```
ws://localhost:8000/ws/chat/<user_id>/
```

- When one user sends a message:
 - The other user receives it instantly
 - The message is stored in the database
-

Message Model

Create a `Message` model with the following fields:

- `sender` (ForeignKey → User)
 - `receiver` (ForeignKey → User)
 - `content` (TextField)
 - `timestamp` (auto-generated)
-

Message History API

Provide a REST API endpoint:

```
GET /api/chat/history/<user_id>/
```

Response requirements:

- Returns all messages between the authenticated user and the given user
 - Messages should be ordered by timestamp (oldest → newest)
-

Security Requirements

- Unauthorized users **must not** connect to WebSockets
- Users cannot listen to chats they are not part of
- Authentication must be validated inside the WebSocket consumer

Bonus (Optional)

These are **not mandatory**, but will be considered a plus:

- Typing indicator
- Online/offline status
- Docker setup
- Basic unit tests
- JWT or token-based authentication
- Message delivery/read status

Time Expectation

- Recommended time: 10-12** hours**
- Maximum allowed: **48*** hours**

Evaluation Criteria

Must-Have

- Correct Django Channels setup
- Secure authentication handling
- Messages persisted correctly
- Clean, readable, maintainable code

Red Flags

- No authentication or permission checks
- Hard-coded user IDs
- Poor project structure
- WebSocket working without message persistence

Submission Guidelines

- Share the project via GitHub or ZIP archive
- Include a brief README with setup instructions
- Ensure the project runs locally

Good luck! This task is designed to assess real-world Django backend knowledge and best practices.